ПРОГРАММИРОВАНИЕ И ОБРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

Классы и объекты. SOLID

Лекция 2, часть 2 — продолжаем с того места, где остановились

Фоменко М.Ю.

МАРШРУТ КУРСА

Где мы находимся

Было

Четыре принципа ООП: инкапсуляция, абстракция, наследование, полиморфизм. Пока — на уровне слов и аналогий.

Сегодня

Из чего физически состоит класс в C# — поля, свойства, конструкторы, static, ссылки. И SOLID: пять правил, чтобы классы не сгнили.

Дальше

Интерфейсы, делегаты и события, DI, паттерны. И первый спринт — скелет сервера, где всё это понадобится.

Классы — это тот кирпич, из которого дальше собирается всё остальное.

1

Классы и объекты

Из чего сделан класс в C# и почему без него код превращается в кашу

ПРОБЛЕМА

Как выглядит код без классов

Храним студентов. Данных стало больше — заводим ещё один массив. И ещё один.

```
string[] names   = { "Иванов", "Петров", "Сидоров" };
int[]    grades  = { 5, 4, 3 };
bool[]   isActive = { true, true, false };

// отчислили Петрова — надо аккуратно удалить из ТРЁХ массивов
// перепутал индекс — и у Иванова оценка Сидорова
// добавили "группу" — правим каждый метод, где эти массивы
```

Данные отдельно, логика отдельно, связь между ними держится только на честном слове программиста.

ОПРЕДЕЛЕНИЕ

Объект

Объект — конкретная сущность, у которой есть состояние, поведение и уникальность.

1 Состояние

То, что объект про себя знает: баланс счёта, номер, владелец. Это данные.

2 Поведение

То, что объект умеет делать: пополнить, снять, показать выписку. Это методы.

3 Уникальность

Два счёта с одинаковым балансом — всё равно два разных счёта.

Объект сам хранит своё состояние и сам им управляет. Именно поэтому данные и логика живут вместе.

ОПРЕДЕЛЕНИЕ

Класс

Класс — шаблон, чертёж. Он описывает, какие данные и какое поведение будут у объектов, созданных по нему.

Класс = чертёж

Один на всех. Существует в коде. По нему ничего нельзя «потрогать» напрямую — это описание, а не вещь.

Объект = экземпляр

Их может быть тысяча. Появляется в памяти в момент new. У каждого — свой набор значений полей.

Каждый объект обязательно принадлежит какому-то классу.

АНАТОМИЯ

Из чего состоит класс

Поля

Переменные внутри класса. Хранят состояние объекта. Обычно private.

Свойства

Контролируемый доступ к полям снаружи: get / set с проверками.

Методы

Функции класса. Описывают поведение — что объект умеет делать.

Конструктор

Особый метод. Вызывается при new и приводит объект в рабочее состояние.

Ещё бывают события, индексаторы, вложенные типы — до них дойдём в следующих лекциях.

код

Первый класс на C#

Сквозной пример на сегодня — банковский счёт.

```
public class BankAccount
{
    private string _owner;      // поле: кто владелец
    private decimal _balance;   // поле: сколько денег

    public void Deposit(decimal amount)   // метод: поведение
    {
        _balance = _balance + amount;
    }

    public decimal GetBalance() => _balance;
}
```

код

Создание объекта: new и точка

```
BankAccount acc = new BankAccount();   // выделили объект в памяти

acc.Deposit(1000);                     // обращаемся через точку
acc.Deposit(500);

Console.WriteLine(acc.GetBalance());   // 1500

BankAccount other = new BankAccount(); // это уже ДРУГОЙ счёт
Console.WriteLine(other.GetBalance()); // 0
```

new создаёт объект в памяти и возвращает ссылку на него. Каждый new — новый, независимый набор полей.

код

Конструктор

Объект не должен рождаться недоделанным: счёт без владельца — мусор.

```
public class BankAccount
{
    private string _owner;
    private decimal _balance;

    public BankAccount(string owner, decimal start)  // конструктор
    {
        _owner = owner;
        _balance = start;
    }
}

var acc = new BankAccount("Иванов", 1000);  // иначе не создать
```

ПРАВИЛО ЯЗЫКА

Конструктор по умолчанию и перегрузка

Не написал ни одного конструктора — компилятор добавит пустой сам. Написал свой — пустого больше нет.

```
public BankAccount(string owner, decimal start)
{
    this.owner = owner;         // this — «текущий объект»
    this.balance = start;       // нужен, когда имена совпали
}

public BankAccount(string owner) : this(owner, 0) { }
// перегрузка: второй конструктор вызывает первый
```

ИНКАПСУЛЯЦИЯ НА ПРАКТИКЕ

Модификаторы доступа

`public`

Видно всем. Это витрина класса — то, чем им пользуются снаружи.

`private`

Видно только внутри самого класса. По умолчанию — всё должно быть таким.

`protected`

Видно внутри класса и его наследникам. Пригодится на следующем слайде про наследование.

Правило по умолчанию: всё private, наружу — только необходимое.

internal (видно внутри сборки) встретится позже, когда проект разрастётся на несколько библиотек.

ДО / ПОСЛЕ

Зачем вообще private

Было: поле наружу

```
public class BankAccount
{
    public decimal Balance;
}

acc.Balance = -100000;
// и никто не помешал
```

Стало: поле спрятано

```
public class BankAccount
{
    private decimal _balance;

    public void Withdraw(decimal a)
    {
        if (a > _balance)
            throw new Exception("нет денег");
        _balance -= a;
    }
}
```

Класс сам отвечает за то, чтобы его состояние оставалось валидным. Снаружи испортить его нельзя.

КОД

Свойства: поле снаружи, метод внутри

Пользователю свойство выглядит как поле, но за ним стоит код.

```
private decimal _balance;

public decimal Balance          // свойство
{
    get { return _balance; }              // чтение
    set
    {
        if (value < 0)                    // value — то, что присвоили
            throw new ArgumentException("баланс не бывает отрицательным");
        _balance = value;
    }
}

acc.Balance = 500;   // выглядит как поле, но проверка сработала
```

КОД

Авто-свойства и свойства только для чтения

```
public string Owner { get; set; }          // авто-свойство:
                                           // поле компилятор создаст сам

public string Number { get; }              // только чтение,
                                           // задаётся в конструкторе

public decimal Balance { get; private set; } // читать всем,
                                             // менять — только классу

public bool IsEmpty => Balance == 0;       // вычисляемое свойство
```

Начинайте с самого закрытого варианта. Открыть доступ позже легко, забрать обратно — почти никогда.

код

Статические члены

static относится к классу в целом, а не к конкретному объекту.

```
public class BankAccount
{
    private static int _totalAccounts;   // один на все объекты

    public BankAccount(string owner)
    {
        _totalAccounts++;
    }

    public static int TotalAccounts => _totalAccounts;
}

Console.WriteLine(BankAccount.TotalAccounts);  // без new, через класс
```

ЗДРАВЫЙ СМЫСЛ

static: когда можно и когда себе дороже

Уместно

- Константы и настройки: Math.PI
- Чистые функции без состояния: Math.Max(a, b)
- Фабричные методы: Account.Create(...)
- Счётчики и статистика по всем объектам

Опасно

- Изменяемое общее состояние — глобальная переменная под другим именем
- «Менеджер всего», который знает про всех
- Такой код почти невозможно протестировать и подменить

Если хочется сделать static, чтобы «удобнее дотянуться» — это не про удобство, а про будущую боль.

КАК ЭТО ЛЕЖИТ В ПАМЯТИ

Значимые и ссылочные типы

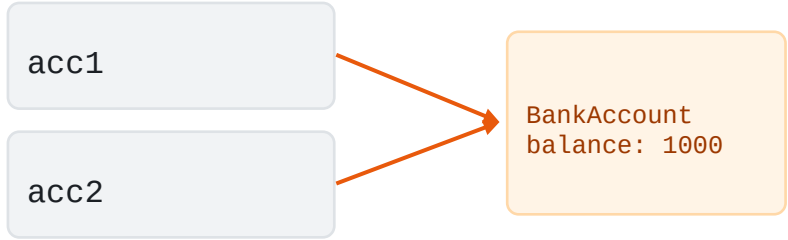
Стек — сами значения

```
int x = 5
```

```
int y = x   →   5
```

Переменная y получила копию. Меняй её сколько угодно — x не изменится.

Куча — сами объекты



Обе переменные держат ссылку на один и тот же объект.

Присваивание ссылочной переменной копирует ссылку, а не объект.

код

Классическая ловушка

```
BankAccount acc1 = new BankAccount("Иванов", 1000);
BankAccount acc2 = acc1;          // скопировали ССЫЛКУ, не объект

acc2.Deposit(500);

Console.WriteLine(acc1.Balance);  // 1500, а не 1000
Console.WriteLine(acc2.Balance);  // 1500

Console.WriteLine(acc1 == acc2);  // True — это один объект
```

Хотите настоящую копию — пишите её руками: new BankAccount(acc1.Owner, acc1.Balance).

СРАВНЕНИЕ

class или struct

	class	struct
Тип	ссылочный	значимый
Где живёт объект	в куче	обычно на стеке
Что копируется	ссылка	всё содержимое
Наследование	есть	нет
Может быть null	да	нет (если не Nullable)
Когда брать	почти всегда	маленькие значения: точка, цвет, деньги

Сомневаетесь — берите class. struct — осознанная оптимизация, а не стиль жизни.

АССОЦИАЦИЯ

Объекты внутри объектов

Полем класса может быть не только int или string, но и другой объект.

```
public class Client
{
    public string Name { get; }
    public List<BankAccount> Accounts { get; } = new();

    public decimal Total()
    {
        decimal sum = 0;
        foreach (var acc in Accounts) sum += acc.Balance;
        return sum;
    }
}
```

Так из отдельных классов вырастает модель предметной области — дерево объектов, а не куча массивов.

ИТОГ ПЕРВОЙ ЧАСТИ

Что должно остаться в голове

1. Класс — чертёж, объект — экземпляр. new создаёт объект и возвращает ссылку.
2. Поля — private. Наружу — свойства и методы, и только то, что действительно нужно.
3. Конструктор существует, чтобы недоделанного объекта не появилось в принципе.
4. static — про класс целиком. Изменяемая статика — это глобальная переменная.
5. Присваивание объекта копирует ссылку. Два имени могут смотреть на один объект.

2

SOLID

Классы писать научились. Теперь — как не превратить их в помойку через полгода

ОПРЕДЕЛЕНИЕ

Что такое SOLID

Пять принципов проектирования, сформулированных Робертом Мартином в начале 2000-х. Акроним придумал Майкл Фэзерс. Это не закон и не синтаксис — это пять способов не выстрелить себе в ногу.

S	O	L	I	D
Single Responsibility	Open / Closed	Liskov Substitution	Interface Segregation	Dependency Inversion

Код — это не слова, это решения проблем. А эти пять терминов — названия лучших решений, придуманных до вас.

ПОСТАНОВКА

Сквозной пример: убогий графический редактор

Пишем редактор фигур. На каждом шаге заказчик приносит новое требование, и код трещит.

```
var shapes = new List<Shape>();          // Shape: Type, X, Y, Radius, Width...

foreach (var shape in shapes)
{
    if (shape.Type == "Circle")
        DrawCircle(shape.X, shape.Y, shape.Radius);
    else if (shape.Type == "Rectangle")
        DrawRectangle(shape.X, shape.Y, shape.Width, shape.Height);
    // а завтра сюда добавят треугольник. И ещё один. И ещё.
}
```

Инкапсуляция, наследование и полиморфизм из прошлой лекции уже почистили классы. Теперь — пять принципов на том же примере.

ПРИНЦИП ЕДИНСТВЕННОЙ ОТВЕТСТВЕННОСТИ

S — SRP

Проблема

Заказчик просит сохранять фигуры в файл. Первая мысль — дописать метод SaveToFile() прямо в Shape. Класс превращается в швейцарский нож, который ломается от любого чиха.

Суть

Один класс — одна причина для изменения. Shape отвечает за геометрию и отрисовку. Сохранение — отдельная задача, значит отдельный класс ShapePersistence.

По-человечески

Делай что-то одно, но делай это хорошо.

ДО / ПОСЛЕ

S — SRP на коде

Было

```
abstract class Shape
{
    public abstract void Draw();

    public void SaveToFile(string p)
    {
        // сериализация, пути,
        // права доступа, ошибки...
    }
}
```

Стало

```
abstract class Shape
{
    public abstract void Draw();
}

class ShapePersistence
{
    public void Save(Shape s, string p)
    { /* только сохранение */ }
}
```

Поменялся формат файла — правим один класс. Появилась новая фигура — Persistence вообще не трогаем.

ПРИНЦИП ОТКРЫТОСТИ / ЗАКРЫТОСТИ

O — OCP

Проблема

Просят добавить треугольник. Если для этого нужно лезть в работающий код редактора и дописывать ветку в if-else — страшно трогать: сломаешь то, что уже работает.

Суть

Класс открыт для расширения и закрыт для изменения. Новый класс Triangle : Shape добавляется рядом, существующий код отрисовки не меняется ни на строчку.

По-человечески

Пишем код как Лего: новую деталь прицепляем, старые не переплавляем.

код

O — OCP на коде

```
// добавляем НОВЫЙ файл — и всё
public class Triangle : Shape
{
    public override void Draw() => Console.WriteLine("Рисую треугольник");
}

// а вот этот код не меняется вообще:
foreach (var shape in shapes)
    shape.Draw();
```

Признак нарушения OCP — длинный switch или if-else по типу объекта. Почти всегда лечится полиморфизмом.

ПРИНЦИП ПОДСТАНОВКИ БАРБАРЫ ЛИСКОВ

L — LSP

Проблема

Классика: делаем Square наследником Rectangle. Формально квадрат — прямоугольник. Но у Rectangle есть Width и Height, которые меняются независимо, а у квадрата так нельзя.

Суть

Если подставить наследника вместо родителя, программа не должна этого заметить. Наследник обязан соблюдать контракт поведения родителя, а не только набор методов.

По-человечески

Наследование — это «ведёт себя как», а не «похоже по словарю».

КАК ЛОМАЕТСЯ

L — LSP на коде

```
class Square : Rectangle
{
    public override int Width  { set { base.Width = base.Height = value; } }
    public override int Height { set { base.Width = base.Height = value; } }
}

void Resize(Rectangle r)          // код ждёт обычный прямоугольник
{
    r.Width = 5;
    r.Height = 10;
    Console.WriteLine(r.Width * r.Height);   // ждём 50, получаем 100
}

Resize(new Square());             // компилируется. И врёт.
```

ПРИНЦИП РАЗДЕЛЕНИЯ ИНТЕРФЕЙСА

I — ISP

Проблема

Просят сделать часть фигур интерактивными — кнопку. Добавляем в Shape методы OnClick() и OnHover(). Теперь у обычного круга болтаются пустые методы, которые никто не вызовет.

Суть

Много маленьких специфичных интерфейсов лучше одного большого. Класс реализует только то, что ему действительно нужно уметь.

По-человечески

Лучше IDrawable и IClickable, чем один IAllMightyGodObject.

КОД

I — ISP на коде

```
interface IDrawable  { void Draw(); }
interface IClickable { void OnClick(); }

class Circle : IDrawable
{
    public void Draw() { }
}

class Button : IDrawable, IClickable   // умеет и то, и другое
{
    public void Draw() { }
    public void OnClick() { }
}
```

Интерфейсы разбираем подробно на следующей лекции — здесь важно само правило: не заставляй класс уметь лишнее.

ПРИНЦИП ИНВЕРСИИ ЗАВИСИМОСТЕЙ

D — DIP

Проблема

Главный класс GraphicEditor сам создаёт внутри себя new Circle() и new Rectangle(). Он жёстко привязан к конкретным классам: появился Triangle — снова лезем внутрь редактора.

Суть

Модули верхнего уровня зависят от абстракций, а не от конкретных классов. Конкретные фигуры передаются извне — через конструктор или метод.

По-человечески

Редактору достаточно знать, что это «фигуры». Какие именно — не его дело.

ДО / ПОСЛЕ

D — DIP на коде

Было: жёсткая привязка

```
class GraphicEditor
{
    public void DrawAll()
    {
        var c = new Circle();
        var r = new Rectangle();
        c.Draw();
        r.Draw();
    }
}
```

Стало: зависимость от абстракции

```
class GraphicEditor
{
    private readonly List<Shape> _shapes;

    public GraphicEditor(List<Shape> s)
    { _shapes = s; }     // извне

    public void DrawAll()
    { foreach (var s in _shapes) s.Draw(); }
}
```

Теперь редактор можно протестировать, подсунув ему любой набор фигур — хоть фиктивных.

ШПАРГАЛКА

SOLID в пяти строках

Принцип	На человеческом языке	Признак нарушения
S — SRP	Класс делает одно дело	Класс на 800 строк с именем Manager
O — OCP	Расширяем, не переписывая	Длинный if-else по типу объекта
L — LSP	Наследник не врёт про поведение	Переопределённый метод, ломающий ожидания
I — ISP	Маленькие интерфейсы	Пустые методы «чтобы компилировалось»
D — DIP	Зависим от абстракций	new конкретного класса в глубине логики

Принципы — инструмент, а не религия. В учебном проекте на три класса SOLID ради SOLID вредит не меньше, чем его отсутствие.

К СЛЕДУЮЩЕМУ ЗАНЯТИЮ

Что дальше

Лекция 3	Интерфейсы, делегаты и события, внедрение зависимостей. Продолжение прямо с буквы D.
К спринту 1	Сущности будущего сервера — это обычные классы: поля private, свойства с проверками, конструктор, который не даёт создать мусор.
Проверить себя	Взять свой код с прошлого семестра и найти в нём хотя бы одно нарушение SRP. Оно там есть.

Вопросы

Классы, объекты, SOLID — спрашивайте, пока свежо